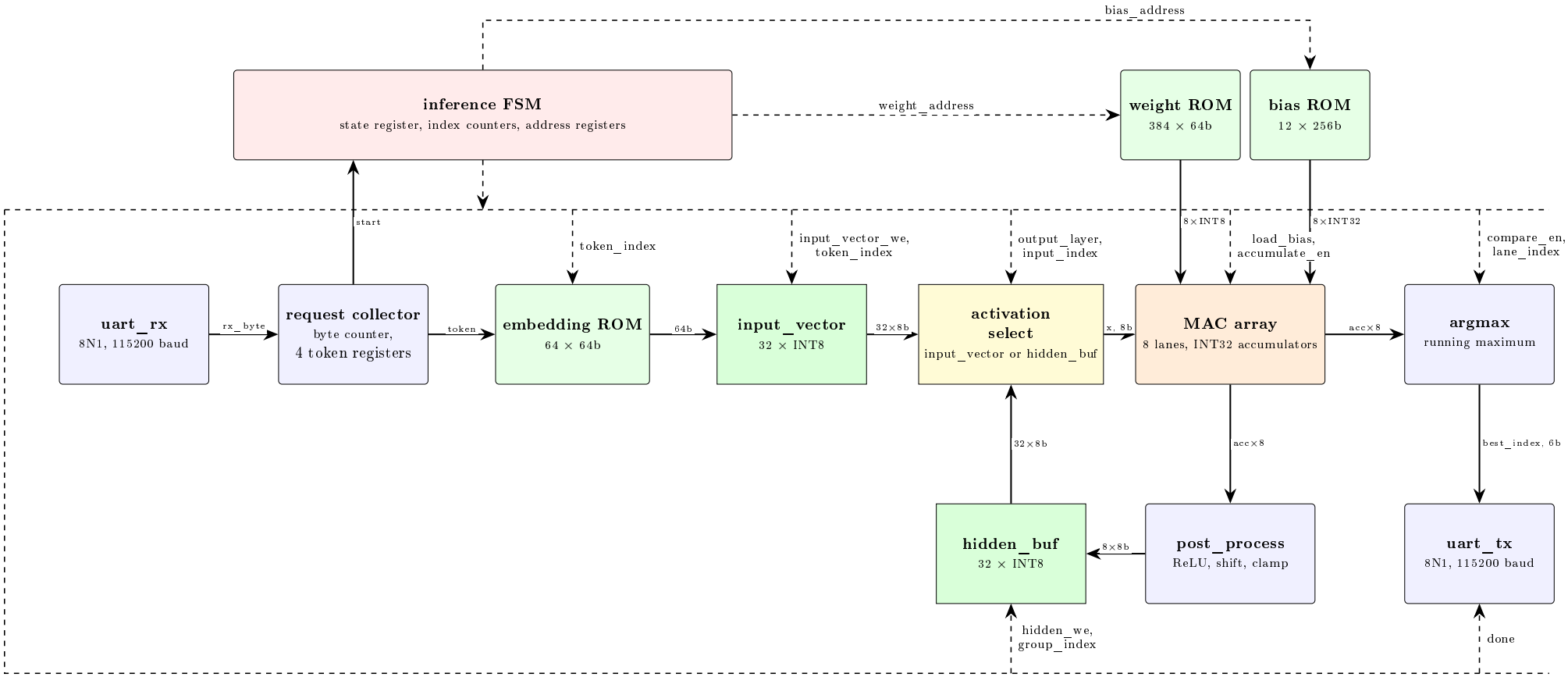
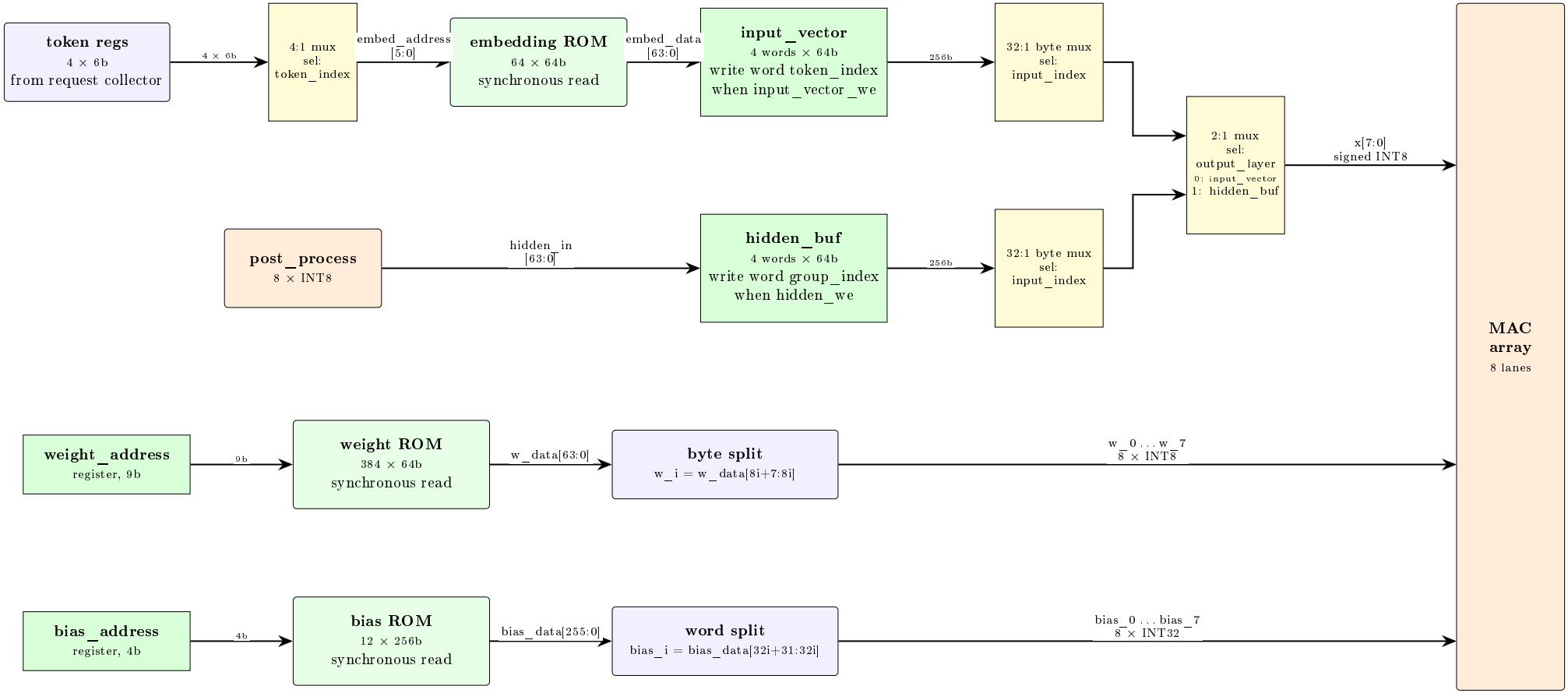
1. High-level schematic

Solid arrows carry data, dashed arrows carry control from the FSM.



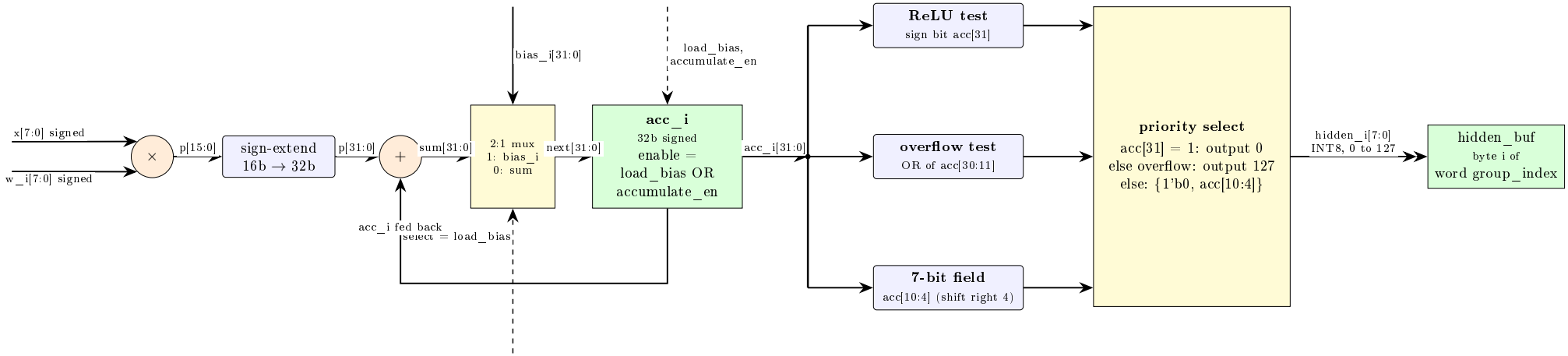
The collector assembles four UART bytes into four 6-bit token registers and raises `start`. The FSM then walks the datapath: embedding lookup into `input_vector`, the hidden layer into `hidden_buf`, the output layer into the argmax, and finally `done` triggers `uart_tx`. The hidden and output layers use the same activation select, ROMs, and MAC array; only `output_layer` changes which buffer feeds the activation select.

2. Low level: input side (embedding, buffers, activation select, ROMs)



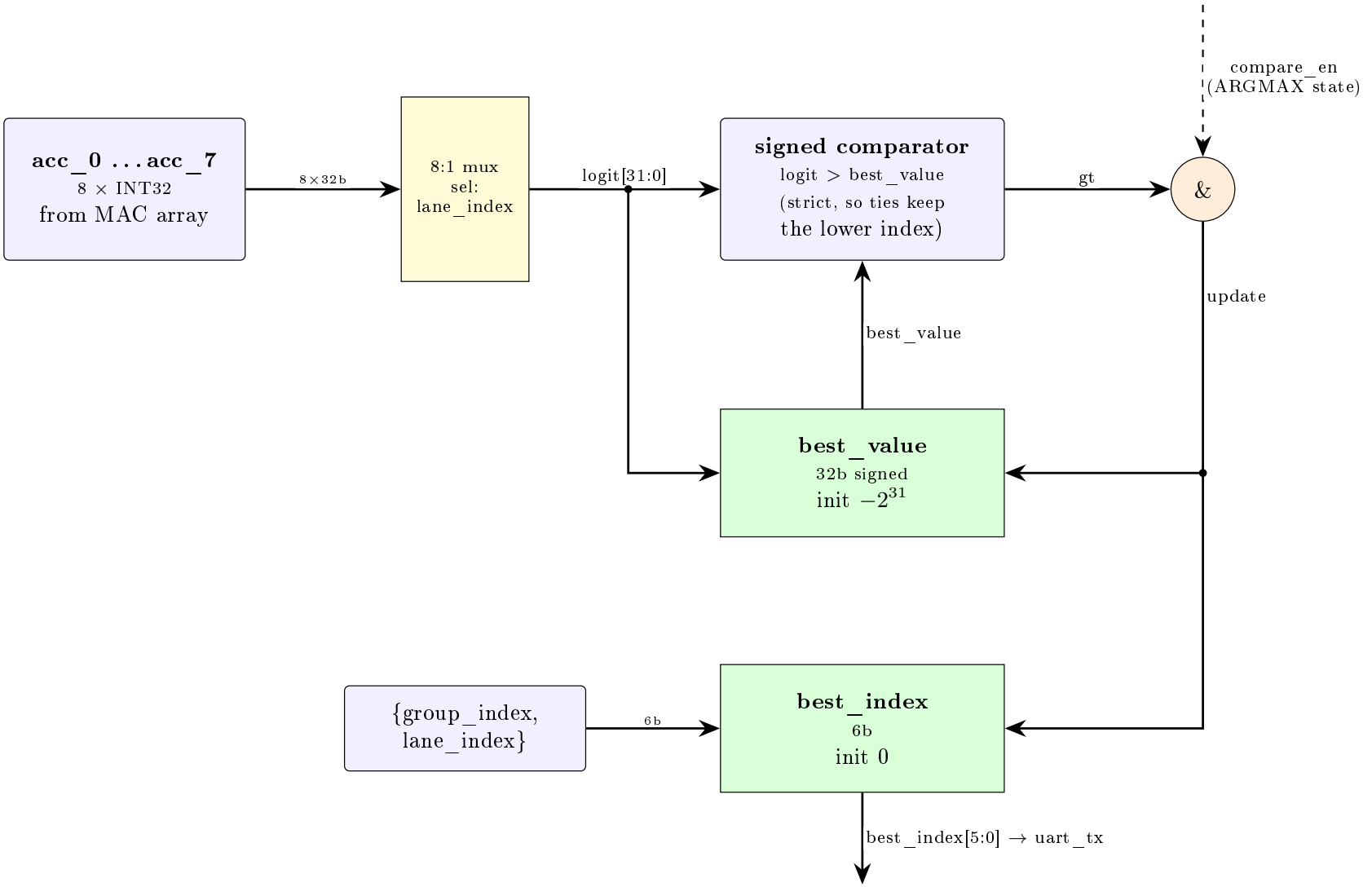
Read timing: an address held in READ is captured at the clock edge and the ROM output is valid during the next state. The byte muxes and the 2:1 mux are combinational, so the activation x is valid in the same cycle `input_index` is stable.

3. Low level: one MAC lane and the post-processing of its accumulator



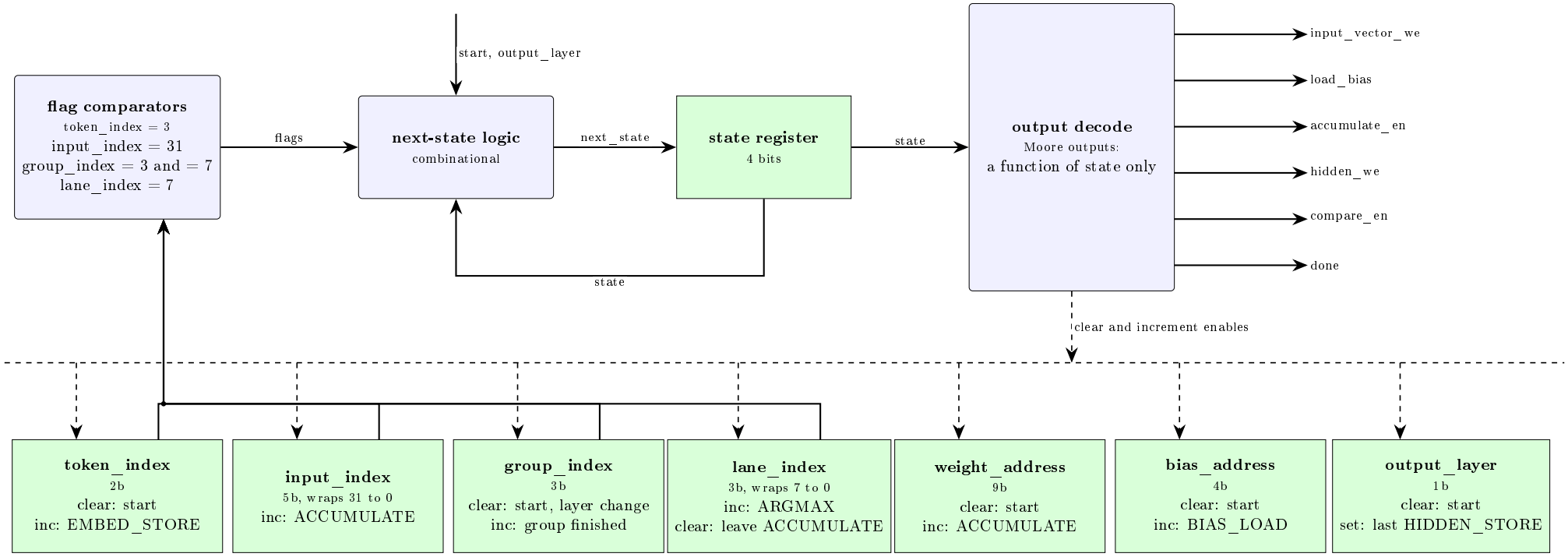
Lanes 1 to 7 are identical copies. They share x, `load_bias`, and `accumulate_en`, and differ only in w_i, $bias_i$, and the accumulator they own.
Why the post-processing is only wiring and one OR: for acc ≥ 0, acc $\gg 4 >$ 127 exactly when any of acc[30:11] is set; otherwise the 7-bit result is acc[10:4].

4. Low level: argmax



At start: best_value ← -2^{31}, best_index ← 0. Logits arrive in increasing token order (group 0 lane 0 first), so the first logit equal to the maximum is the one recorded, which gives the lowest-index tie rule.

5. Low level: control (state register, flags, counters, output decode)



Control output	Asserted in state, and what it enables
input_vector_we	EMBED_STORE: writes embed_data into input_vector word token_index
load_bias	BIAS_LOAD: every accumulator loads its bias
accumulate_en	ACCUMULATE: every accumulator adds $x \cdot w_i$
hidden_we	HIDDEN_STORE: writes the 8 post-processed bytes into hidden_buf word group_index
compare_en	ARGMAX: lets the comparator update best_value and best_index
done	DONE: starts uart_tx with best_index